Local Agentic Programming on the Cheap: Claude Code + Ollama + Gemma4

kdnuggets.com/local-agentic-programming-on-the-cheap-claude-code-ollama-gemma4

Shittu Olumide

Introduction

Visualize this: a multi-agent workflow that reads files, writes patches, runs tests, and iterates across four services, making 400 API calls in a single afternoon. The notification arrives. You have crossed the soft limit again. Every token costs money, every prompt sends your proprietary code to a third-party server, and the rate limits interrupt long-running sessions — the only solution is paying more.

Gemma 4 26B MoE activates only 3.8 billion of its 26 billion parameters per forward pass. It scores 77.1% on **LiveCodeBench** v6 and 86.4% on **τ2-bench** agentic tool use — the benchmark that specifically tests what happens when a model has to call tools, execute steps, and handle errors across a multi-step workflow. The previous generation, Gemma 3 27B, scored 6.6% on that same benchmark. That is not a small upgrade. It is the difference between a model that cannot reliably call tools and one that can run a Claude Code agentic loop without constantly malforming its function call parameters.

This article builds the full stack: Ollama serving Gemma 4 locally, the Modelfile that prevents context window failures in agentic sessions, the `settings.json` that wires Claude Code to the local endpoint, a verification script that confirms everything is working before you use it on real code, and an honest rundown of what breaks and how to fix it. The audience is engineers who already understand what large language models (LLMs) are and what agentic loops cost. No hand-holding on the basics.

Why Gemma 4?

Released on April 2, 2026 under Apache 2.0, Gemma 4 is Google DeepMind's most capable open-weight model family to date. Four variants shipped: E2B (2B effective), E4B (4B effective), 26B MoE, and 31B Dense. The 26B MoE uses 128 small experts and activates only 8 per token plus one shared expert, delivering near-31B quality at dramatically lower compute cost.

Previous Gemma versions used a custom Google license with commercial use restrictions ambiguous enough that enterprise legal teams routinely flagged it as a blocker. Gemma 4 is Apache 2.0, a first for the Gemma family. If your team wants to embed this in internal

tooling, ship products on top of it, or run it in production pipelines without legal review overhead, that change matters operationally.

// The Numbers That Matter for Coding Agents

Benchmark	Gemma 3 27B	Gemma 4 26B MoE	Gemma 4 31B Dense
τ2-bench (agentic tool use)	6.6%	~79%	86.4%
LiveCodeBench v6	29.1%	77.1%	80.0%
GPQA Diamond	42.4%	82.3%	84.3%
AIME 2026 (math)	20.8%	88.3%	89.2%
Arena AI ELO	1365	1441	1452

// Hardware Requirements

Before pulling an 18 GB model, know what you are actually working with. The Gemma 4 family was designed to span edge devices through workstations, and the four variants reflect that range.

Variant	Ollama tag	Active params	VRAM at Q4	Context window
Edge 4B	gemma4:e4b	4B	~6 GB	128K
26B MoE	gemma4:26b	3.8B	~16–18 GB	256K
31B Dense	gemma4:31b	31B	~24–32 GB	256K

// Installing Ollama, Gemma 4, and Claude Code

Step 1: Install Ollama

```
# macOS and Linux -- one-line install
curl -fsSL https://ollama.com/install.sh | sh

# Verify version -- must be 0.14.0+ for Anthropic Messages API support
# The Anthropic-compatible endpoint was added in January 2026
ollama version
# Expected: ollama version is 0.22.x or higher (as of May 2026)

# Windows: download the native installer from https://ollama.com
# WSL2 is recommended if you want GPU passthrough on Windows
```

After installation, Ollama starts as a background service on port 11434. Verify it is up:

```
curl http://localhost:11434
# Expected response: Ollama is running
```

Step 2: Pull Gemma 4

```
# The 26B MoE -- recommended for this setup (~18 GB download)
ollama pull gemma4:26b

# While you wait, confirm the download is progressing
ollama ps
# Shows currently downloading or running models

# Optional: also pull the 31B for comparison on capable hardware
ollama pull gemma4:31b

# Confirm the pull completed
ollama list
# Should show gemma4:26b with size and modification date
```

Step 3: Install Claude Code

```
# Prerequisites: Node.js 18 or later
node --version   # Confirm you are on 18+

# Install Claude Code CLI globally
npm install -g @anthropic-ai/claude-code

# Verify the install
claude --version
```

With Ollama running and Gemma 4 pulled, the natural next instinct is to export the environment variables and launch Claude Code immediately.

The Modelfile

Ollama's default context window for Gemma 4 is 4K tokens. Gemma 4's actual context window is **128K–256K**. That 4K default is not a suggestion — it is what Ollama will use unless you override it. In a Claude Code agentic session that reads source files, holds conversation history, and maintains tool call results across multiple turns, 4K tokens is exhausted in seconds.

Without the context override, Claude Code loses track of file contents mid-edit, forgets earlier instructions, and produces fragmented changes. Specifically: when an agent tries to refactor a 200-line service class, it cleanly forgets the second half exists. The agent does not raise an error. It just silently works on an incomplete view of the file and produces partially correct output that breaks downstream.

The fix is a Modelfile that bakes the correct context size and other inference parameters into a named model variant. Create this file:

```
# ~/.ollama/Modelfiles/gemma4-claude
# Gemma 4 26B MoE variant tuned for Claude Code agentic sessions.
# Bakes context window, temperature, and system prompt into the model
# so every Claude Code session starts with the correct configuration.
#
# Build with:
#   mkdir -p ~/.ollama/Modelfiles
#   ollama create gemma4-claude -f ~/.ollama/Modelfiles/gemma4-claude

FROM gemma4:26b

# Context window -- 65536 tokens (64K) is the tested-safe floor for real
# codebases without triggering swap on 16-18 GB VRAM systems.
# Increase to 131072 (128K) if you have headroom on 24 GB+ systems.
# Do not go above 131072 unless you have profiled your memory usage
# under load -- Ollama pre-allocates the full KV cache upfront.
PARAMETER num_ctx 65536

# Temperature -- 0.2 is deliberately low for agentic coding.
# Higher temperature introduces variability in tool call parameter
# formatting that causes Claude Code's tool validator to reject calls.
# For creative tasks, you would set this higher. For agentic loops: low.
PARAMETER temperature 0.2

# top_p -- nucleus sampling threshold. 0.9 keeps generation focused
# while avoiding the repetition loops that top_p=1.0 can produce on
# long agentic sessions.
PARAMETER top_p 0.9

# repeat_penalty -- penalizes the model for repeating tokens.
# 1.15 helps prevent tool call loops where Gemma 4 retries the same
# failed tool call with nearly identical parameters indefinitely.
PARAMETER repeat_penalty 1.15

# num_predict -- maximum tokens per response. 4096 is sufficient for
# most code patches. Increase to 8192 if you regularly generate
# large files in a single generation.
PARAMETER num_predict 4096

# System prompt -- reinforces coding agent behavior and explicit
# tool use discipline. Gemma 4 benefits from being reminded to
# commit to tool calls rather than describing what it would do.
SYSTEM """You are a senior software engineer operating as a coding agent.

When working with code:
- Read files before editing them. Never assume file contents.
- Make one focused change at a time and verify it before proceeding.
- When a tool call fails, examine the error carefully before retrying.
  Do not retry with identical parameters. Diagnose first.
```

```
- Prefer surgical edits over full file rewrites.
- Run tests after each meaningful change, not after a batch of changes.
- If you are uncertain about the codebase structure, read more files
  rather than guessing.

Be precise and methodical. Avoid explaining what you are about to do
when you could simply do it."""
```

Build the variant:

```
# Create the Modelfiles directory if it does not exist
mkdir -p ~/.ollama/Modelfiles

# Save the Modelfile content from above to this path, then build:
ollama create gemma4-claude -f ~/.ollama/Modelfiles/gemma4-claude

# Verify the variant was created
ollama list
# Should show gemma4-claude alongside gemma4:26b

# Quick smoke test -- verify it loads and responds
ollama run gemma4-claude "What is the time complexity of binary search and why?"
# Expect a clear, concise technical response within a few seconds
```

Wiring Claude Code to the Local Model

With the model variant built, the configuration layer connects Claude Code to Ollama. Two environment variables are the core of this, but three additional variables prevent the most common failure modes.

Ollama's Anthropic-compatible endpoint is at `http://localhost:11434`, not `http://localhost:11434/v1`. The `/v1` path is Ollama's OpenAI-compatible layer. Claude Code uses the Anthropic Messages API protocol, which maps to the root endpoint. Using the `/v1` path will produce authentication errors or unexpected behavior.

// Global Settings — ~/.claude/settings.json

This configuration applies to every Claude Code session across all projects. It is the right choice unless you are switching between local and cloud models frequently per project.

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:11434",

    "ANTHROPIC_AUTH_TOKEN": "ollama",

    "ANTHROPIC_API_KEY": "",

    "ANTHROPIC_MODEL": "gemma4-claude",
```

```
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "gemma4-claude",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "gemma4-claude",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "gemma4-claude",

    "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1"
  }
}
```

Why each variable matters:

- **ANTHROPIC_BASE_URL** redirects all Claude Code API calls from Anthropic's servers to your local Ollama instance.
- **ANTHROPIC_AUTH_TOKEN** must be set to any non-empty string; Ollama ignores the value but Claude Code requires the header to be present.
- **ANTHROPIC_API_KEY: ""** explicitly empties the key so Claude Code cannot fall back to a real Anthropic API key if one happens to be set in your shell environment. Without this, a misconfigured `ANTHROPIC_BASE_URL` might silently fail over to the paid API.
- **ANTHROPIC_MODEL** is the primary model name Claude Code sends in requests. Set this to your custom Modelfile variant, `gemma4-claude` not `gemma4:26b`. The raw model tag does not carry the context window override.
- **ANTHROPIC_DEFAULT_SONNET_MODEL**, **ANTHROPIC_DEFAULT_HAIKU_MODEL**, and **ANTHROPIC_DEFAULT_OPUS_MODEL**: Claude Code internally routes different task types to different model tiers. Setting all three to the same local model ensures every request lands at your Ollama instance regardless of which tier Claude Code internally selects.
- **CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS: "1"** strips the Anthropic-specific beta headers that Claude Code adds to requests. Local inference servers do not recognize these headers and reject requests that include them. Setting this variable prevents that error without affecting any core Claude Code functionality.

// Per-Project Configuration — .claude/settings.json

For projects where you want local inference isolated from your global setup — private repositories, sensitive codebases, or projects with specific model requirements — use a project-level settings file instead:

```
# In your project root
mkdir -p .claude

cat > .claude/settings.json << 'EOF'
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:11434",
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_API_KEY": "",
    "ANTHROPIC_MODEL": "gemma4-claude",
```

```
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "gemma4-claude",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "gemma4-claude",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "gemma4-claude",
    "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1"
  }
}
EOF
```

Claude Code reads the project-level `.claude/settings.json` when it exists, overriding global settings for that project. Add `.claude/settings.json` to your `.gitignore` if the settings contain anything environment-specific, or commit it if you want the entire team running local inference on that project.

// Verifying the Setup

Before running Claude Code against a real codebase, verify three things: Ollama is serving correctly, the model responds to API calls in the Anthropic Messages format, and tool calling specifically works. The third point is non-negotiable: tool calling is how Claude Code reads files, writes patches, and executes commands. A model that cannot format tool calls correctly will loop and fail on basic agentic tasks.

Prerequisites:

```
pip install httpx   # Async HTTP client for the verification script
```

The full verification script:

```
#!/usr/bin/env python3
"""
verify_local_setup.py

Verifies the full Claude Code + Ollama + Gemma 4 stack before use.
Runs three checks in sequence:
  1. Ollama health and model availability
  2. Basic Anthropic Messages API call
  3. Tool calling round-trip

Prerequisites:
  pip install httpx

How to run:
  python verify_local_setup.py

Expected output on a working setup:
  [PASS] Ollama is running on localhost:11434
  [PASS] Model 'gemma4-claude' is available
  [PASS] Anthropic Messages API call successful
  [PASS] Tool calling: model produced a valid tool_use block
  All checks passed -- Claude Code + Ollama + Gemma 4 is ready.
"""
```

```
import httpx
import json
import sys

# ── Configuration ──────────────────────────────────────────────────────────
OLLAMA_BASE_URL = "http://localhost:11434"
MODEL_NAME      = "gemma4-claude"   # Must match your Modelfile variant name
TIMEOUT         = 120.0             # Seconds -- generation can be slow on first cal


def check_ollama_health() -> bool:
    """
    Check 1: Verify Ollama is running and responding.
    Hits the root endpoint which returns 'Ollama is running' when healthy.
    """
    print("\nCheck 1: Ollama health")
    try:
        response = httpx.get(OLLAMA_BASE_URL, timeout=5.0)
        if "Ollama is running" in response.text:
            print(f"  [PASS] Ollama is running on {OLLAMA_BASE_URL}")
            return True
        else:
            print(f"  [FAIL] Unexpected response: {response.text[:100]}")
            return False
    except httpx.ConnectError:
        print(f"  [FAIL] Cannot connect to {OLLAMA_BASE_URL}")
        print("         Is Ollama running? Try: ollama serve")
        return False


def check_model_available() -> bool:
    """
    Check 2: Verify the specific model variant is available in Ollama.
    Uses the /api/tags endpoint which lists all pulled models.
    """
    print("\nCheck 2: Model availability")
    try:
        response = httpx.get(f"{OLLAMA_BASE_URL}/api/tags", timeout=5.0)
        data     = response.json()
        models   = [m["name"] for m in data.get("models", [])]

        # Normalize: Ollama may add ":latest" if not specified
        normalized = [m.split(":")[0] for m in models]

        if MODEL_NAME in models or MODEL_NAME in normalized:
            print(f"  [PASS] Model '{MODEL_NAME}' is available")
            return True
        else:
            print(f"  [FAIL] Model '{MODEL_NAME}' not found")
            print(f"         Available models: {', '.join(models) or 'none'}")
            print(f"         Run: ollama create {MODEL_NAME} -f ~/.ollama/Modelfiles
```

```
            return False
    except Exception as e:
        print(f"  [FAIL] Error checking model list: {e}")
        return False


def check_messages_api() -> bool:
    """
    Check 3: Send a basic Anthropic Messages API call to the local endpoint.
    Verifies the request format, model routing, and basic generation work.
    Uses the same /v1/messages path and request schema that Claude Code uses.
    Note: Claude Code uses http://localhost:11434 (root), not /v1.
    The Anthropic-compatible API is at /api/chat or the root -- Ollama routes it.
    """
    print("\nCheck 3: Anthropic Messages API call")

    payload = {
        "model": MODEL_NAME,
        "max_tokens": 100,
        "messages": [
            {
                "role": "user",
                "content": "Reply with exactly: VERIFICATION_OK"
            }
        ]
    }

    headers = {
        "Content-Type":      "application/json",
        "x-api-key":         "ollama",            # Required by the API spec; value
        "anthropic-version": "2023-06-01"         # Required version header
    }

    try:
        response = httpx.post(
            f"{OLLAMA_BASE_URL}/v1/messages",
            json=payload,
            headers=headers,
            timeout=TIMEOUT
        )

        if response.status_code != 200:
            print(f"  [FAIL] HTTP {response.status_code}: {response.text[:200]}")
            return False

        data = response.json()

        # Anthropic Messages API response structure:
        # { "content": [{"type": "text", "text": "..."}], "stop_reason": "..." }
        content_blocks = data.get("content", [])
        text_blocks    = [b for b in content_blocks if b.get("type") == "text"]
```

```
        if not text_blocks:
            print(f"  [FAIL] No text content in response: {json.dumps(data, indent=2
            return False

        response_text = text_blocks[0].get("text", "")
        print(f"  [PASS] Anthropic Messages API call successful")
        print(f"         Model response: {response_text[:80]}")
        return True

    except Exception as e:
        print(f"  [FAIL] Request failed: {e}")
        return False


def check_tool_calling() -> bool:
    """
    Check 4: Verify tool calling works end-to-end.
    This is the most important check for Claude Code agentic use.
    Claude Code relies on the model correctly producing tool_use blocks
    for every file operation, shell command, and code execution.

    Sends a simple tool definition and a prompt that should trigger it.
    Verifies the model returns a tool_use block (not just text describing the call).
    """
    print("\nCheck 4: Tool calling verification")

    # A minimal tool definition using the Anthropic function calling schema
    tools = [
        {
            "name": "read_file",
            "description": "Read the contents of a file at the given path.",
            "input_schema": {
                "type": "object",
                "properties": {
                    "path": {
                        "type": "string",
                        "description": "The absolute or relative file path to read"
                    }
                },
                "required": ["path"]
            }
        }
    ]

    payload = {
        "model": MODEL_NAME,
        "max_tokens": 256,
        "tools": tools,
        # Force the model to call a tool rather than respond in text.
        # tool_choice: {"type": "any"} requires any tool use.
        # Remove this if testing whether the model self-selects tools.
        "tool_choice": {"type": "any"},
```

```
    "messages": [
        {
            "role": "user",
            "content": "Read the file at /tmp/test.py and show me its contents."
        }
    ]
}

headers = {
    "Content-Type":      "application/json",
    "x-api-key":         "ollama",
    "anthropic-version": "2023-06-01"
}

try:
    response = httpx.post(
        f"{OLLAMA_BASE_URL}/v1/messages",
        json=payload,
        headers=headers,
        timeout=TIMEOUT
    )

    if response.status_code != 200:
        print(f"  [FAIL] HTTP {response.status_code}: {response.text[:200]}")
        return False

    data           = response.json()
    content_blocks = data.get("content", [])
    tool_blocks    = [b for b in content_blocks if b.get("type") == "tool_use"]

    if not tool_blocks:
        print("  [FAIL] Model did not produce a tool_use block")
        print("         This means tool calling is not working correctly.")
        print("         Agentic Claude Code sessions will fail on file operatior
        print(f"         Full response: {json.dumps(data, indent=2)}")
        return False

    tool_call  = tool_blocks[0]
    tool_name  = tool_call.get("name", "")
    tool_input = tool_call.get("input", {})

    print(f"  [PASS] Tool calling: model produced a valid tool_use block")
    print(f"         Tool called: {tool_name}")
    print(f"         Parameters:  {json.dumps(tool_input)}")

    # Sanity check: did it call the right tool with the right parameter?
    if tool_name == "read_file" and "path" in tool_input:
        print(f"         Tool name and parameter are correct.")
    else:
        print(f"         WARNING: Unexpected tool name or missing 'path' paramet
        print(f"         The model called a tool but not the expected one.")
```

```
        return True

    except Exception as e:
        print(f"  [FAIL] Request failed: {e}")
        return False


def main():
    print("=" * 60)
    print("Claude Code + Ollama + Gemma 4 Setup Verification")
    print("=" * 60)

    checks = [
        check_ollama_health,
        check_model_available,
        check_messages_api,
        check_tool_calling,
    ]

    results = [check() for check in checks]

    print("\n" + "=" * 60)
    passed = sum(results)
    total  = len(results)

    if all(results):
        print(f"All {total} checks passed.")
        print("Claude Code + Ollama + Gemma 4 is ready.")
        print(f"\nLaunch with: claude")
        sys.exit(0)
    else:
        failed_checks = [i + 1 for i, r in enumerate(results) if not r]
        print(f"{passed}/{total} checks passed. Failed: {failed_checks}")
        print("Resolve the failures above before using Claude Code locally.")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

How to run:

```
pip install httpx
python verify_local_setup.py
```

Agentic Task Walkthrough

With verification passing, here is what a real agentic session looks like. The task: take an existing Python module with no tests, analyze it, write a test suite, run the tests, and fix any failures.

```
# Navigate to a project directory
cd ~/projects/my-service

# Confirm Claude Code sees the local configuration
claude --version
# Verify it does not prompt for an Anthropic API key -- if it does,
# the settings.json is not being read correctly

# Start an agentic session
claude

# Inside Claude Code, give the agent a concrete task:
# > Analyze the UserService class in src/user_service.py.
# > Write a pytest test suite covering all public methods.
# > Run the tests and fix any failures.
# > The goal is a clean pytest run with no skips.
```

What the Claude Code tool call trace looks like during this session:

```
→ read_file("src/user_service.py")
  Reading 247 lines...

→ list_files("src/")
  Found: user_service.py, models.py, db.py, exceptions.py

→ read_file("src/models.py")
  Reading 89 lines...

→ write_file("tests/test_user_service.py", [test content])
  Written: 312 lines

→ bash("python -m pytest tests/test_user_service.py -v 2>&1")
  Running 14 tests...
  FAILED tests/test_user_service.py::test_update_email_invalid
    AssertionError: Expected ValidationError, got None

→ read_file("src/user_service.py")  [targeted re-read of update_email method]
  ...

→ write_file("tests/test_user_service.py", [corrected test])
  Patched test_update_email_invalid assertion

→ bash("python -m pytest tests/test_user_service.py -v 2>&1")
  14 passed in 1.23s
```

Gemma 4 handles this pattern reliably — reading files before editing, running tests after changes, and diagnosing failures from error output rather than retrying blindly. The behavior on complex architectural decisions across many files is where cloud models still have an edge. For the task above (analysis, test generation, and targeted fixes), the local setup is fully capable.

What to watch for: If you see the agent produce "Invalid tool parameters" errors and then retry with the same parameters repeatedly, the temperature is too high, or the model is not using the `gemma4-claude` Modelfile variant. Both temperature and the context window override are baked into the variant; the raw `gemma4:26b` tag does not carry them.

// What Breaks and How to Fix It

1. **Tool Parameter Formatting Errors**
   - **Symptom**: Claude Code reports *Invalid tool parameters* repeatedly. The agent apologizes and retries with identical or nearly identical parameters, then loops.
   - **Cause**: This is documented in the **Ollama GitHub issues**. The model produces tool call JSON that does not match the schema Claude Code expects. Most commonly: wrong field names, missing required fields, or nested objects where scalars are expected.
   - **Fix**: Confirm you are running `gemma4-claude` (the Modelfile variant) not `gemma4:26b` directly. The `temperature: 0.2` and system prompt in the Modelfile significantly reduce this. If the issue persists, drop the temperature to 0.1 in the Modelfile and rebuild.
2. **Context Window Swapping to Disk**
   - **Symptom**: Generation slows to a crawl after several turns. `ollama ps` shows GPU utilization dropping. The OS is paging the KV cache to disk.
   - **Fix**:

```
# Option 1: Reduce context window in the Modelfile
# Edit ~/.ollama/Modelfiles/gemma4-claude
# Change: PARAMETER num_ctx 65536
# To:     PARAMETER num_ctx 32768
# Then rebuild: ollama create gemma4-claude -f ~/.ollama/Modelfiles/gemma4-c

# Option 2: Enable KV cache quantization to reduce memory footprint
export OLLAMA_KV_CACHE_TYPE=q8_0
# This quantizes the KV cache itself, reducing memory at a small quality cos
# Restart Ollama after setting this: pkill ollama && ollama serve
```

3. **Model Unloading Between Agent Turns**
   - **Symptom**: Noticeable cold-start delay at the beginning of each Claude Code message. Ollama is unloading the model after an inactivity timeout and reloading it for each request.
   - **Fix**:

```
# Keep the model loaded indefinitely during your work session
export OLLAMA_KEEP_ALIVE=-1

# Or set it in your shell profile for permanent effect
echo 'export OLLAMA_KEEP_ALIVE=-1' >> ~/.zshrc

# Alternatively, use the Ollama API to pin the model
curl http://localhost:11434/api/generate \
  -d '{"model": "gemma4-claude", "keep_alive": -1}'
# This pins the model until you explicitly unload it or restart Ollama
```

4. **Beta Header Rejection Errors**
   - **Symptom**: Claude Code produces *Unexpected value(s) for the anthropic-beta header* errors on launch or mid-session.
   - **Fix**: Confirm `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS: "1"` is in your `settings.json`. If you set it via shell export instead of `settings.json`, verify it is exported in the same shell session where `claude` is running:

```
echo $CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS
# Must print: 1
```

Wrapping Up

The stack described in this article is not a proof of concept. It is a working production configuration that engineers have been running daily since Ollama added Anthropic Messages API support in January 2026. The Modelfile is not optional; it is the difference between a tool that works and one that silently produces incomplete outputs on multi-file tasks. The verification script catches configuration issues before they surface mid-session as confusing agent failures.

The setup built in this article is a private, zero-per-token-cost coding agent that handles the majority of daily engineering tasks — code analysis, test generation, targeted refactoring, and debugging — at generation speeds that are usable on modern hardware.

This setup is not a replacement for cloud inference on complex architectural reasoning across large codebases or SWE-bench class tasks that require deep repository understanding at scale.